

License to Scan

Natajah Roberts

*Co-Editor of License to Scan
Company*

*License to Scan – A License Plate
Detection Company*
Portland, Oregon USA
nrober05@syr.edu

Tyler Gabriele

*Co-Lead Coder of License to
Scan Company*

*License to Scan – A License Plate
Detection Company*
Syracuse, NY USA
tygabrie@@syr.edu

Oluwaseyi Durosinmi-Etti

*Co-Lead Coder of License to
Scan Company*

*License to Scan – A License Plate
Detection Company*
New York, NY USA
odurosin@syr.edu

Richard Dickson

*Co-Editor of License to Scan
Company*

*License to Scan – A License Plate
Detection Company*
Dallas, Texas USA
rbdickso@syr.edu

Abstract—This paper presents a two-stage Automatic License Plate Recognition (ALPR) system combining YOLOv8-based object detection with EasyOCR-based character recognition for real-time license plate identification. Three YOLOv8 model variants were trained and evaluated on a dataset of 9,100 labeled vehicle images. (YOLOv8n, YOLOv8s, and YOLOv8m) The YOLOv8-Nano model trained for 30 epochs and achieved a final validation precision of 0.984, a recall of 0.944, and mAP50 of 0.963.

A three-pass preprocessing pipeline was applied with grayscale conversion, along with Contrast Limited Adaptive Histogram Equalization (CLAHE) contrast normalization, and adaptive thresholding which was developed to improve Optical Character Recognition (OCR) efficiency across varying lighting and image quality conditions. EasyOCR was selected as the character recognition engine with an alphanumeric allowlist and regex-based plate pattern validation. Results demonstrated strong detection performance on both the validation set and a couple of real-world vehicle images, with OCR accuracy becoming an ongoing challenge under low resolution and variable lighting conditions, which was consistent with the findings reported in recent ALPR research within the industry.

Keywords—automatic license plate recognition, YOLOv8, object detection, optical character recognition, EasyOCR, convolutional neural network, deep learning, image preprocessing, real-time detection, toll system automation

I. INTRODUCTION

Methods for detecting and deciphering license plate information from images have grown rapidly in recent years. Systems that rely on accurate license plate detection are widely used across a range of industries and public services, including highway toll collection, law enforcement surveillance, parking management, and traffic monitoring, and the demand for these systems has accelerated the shift from traditional hand-engineered image processing methods toward deep learning architectures that are more robust, faster, and generalizable across real-world conditions. Among the most widely adopted methods for license plate detection today are single-stage object detection models, with the YOLO (You Only Look Once) family of convolutional neural networks representing the current state of the art, for real-time detection tasks. For the character recognition stage that follows detection, Optical Character Recognition (OCR) engines such as EasyOCR, PaddleOCR, and Tesseract have emerged as the primary tools being evaluated and compared in recent literature [1][3]. This research will report on a test of YOLOv8 for license plate detection and EasyOCR for character recognition using a dataset of 9,100 labeled vehicle

images sourced from Kaggle. Model comparisons and visualizations are provided to display the accuracy, precision, and recall of various workflows. Finally, a fine-tuned and optimized model was tested to weigh the potential benefits and drawbacks of certain components in the workflow. In closing, this research discusses the strengths and limitations of the components tested, the challenges encountered during the OCR stage, and the directions that future work should take to close the remaining gaps in ALPR system performance.

II. LITERATURE REVIEW

A. The Field of License Plate Detection

License plate detection is the process of identifying and extracting license plate information, with the use of camera footage and high-level machine learning techniques. There are significant benefits for society in having a good vehicle identification system, including safety, trade, transit, and law enforcement [1]. The introductory systems of Automated License Plate Reader (ALPR) relied heavily on manually crafted features like edge detection and morphological operations, which performed well in controlled environments but struggled when there was inclement weather, variable lighting, occlusion, or variation in the plate's orientation. [1] A recent publication in Scientific Reports addressed these factors and limitations surrounding ALPR. The paper proposed a new approach for the current process in recognizing and detecting license plates by combining the use of YOLOv8 and OCR. With YOLO finding the license plate, OCR techniques can then be used in conjunction with YOLO by accurately reading the characters within the license plate. The integration of these two deep learning methods provides a winning combination for real-time application, speed, and accuracy. [1]

B. YOLO Models

The technology surrounding license plate detection has greatly advanced since its early systems were put in place. Speed and accuracy have improved dramatically across different YOLO algorithms, including YOLOv3, YOLOv8, and YOLOv8 variants. [2] When evaluating these different architectures, it's important to weigh the advantages and disadvantages that each model brings, considering their different approaches for feature extraction, predictive techniques, and efficiency. A prior study that was reviewed by Shyaa and Hasim also found that YOLOv8 has shown a reduced inference time by more than 30% in some cases, while improving license plate detection accuracy. [2] This has especially been the case when reading images with small or

overlapping objects where YOLOv3 has underperformed in comparison to YOLOv8n.

During that same study by Shyaa and Hasim, they directly compared YOLOv8 with Faster R-CNN and SSD on license plate detection. A small dataset of 453 images was coupled with a large dataset of over 8,000 images for a model performance comparison; specifically, how dataset scale can influence the results. One of the biggest takeaways from this study was that YOLOv8 can detect plates at a greater distance, with a wider range of lighting conditions, and across multiple frames when being trained with the larger dataset. [2] Therefore, the smaller dataset of 453 images was outperformed when competing with the larger dataset.

C. OCR Models

The findings by Scientific Reports, along with the research conducted by Shyaa and Hashim, support the notion that license plate recognition is most efficiently completed as a two-step process. Shyaa and Hashim's research strongly advocates that YOLOv8 is most effective architecture for detecting the license plate region, image by image.[2] This only accomplishes the first stage for any ALPR system; therefore, it is worth examining the challenges that are presented in accomplishing the crucial second stage to determine which OCR is the most appropriate for completing the task. These Optical Character Recognition (OCR) engines perform a process of following a structured pipeline. The first process is for the OCR to digest the vehicle image, which leads to the OCR segmenting the individual characters on the plate internally. The final process of the OCR is completed by pattern matching to identify each character of the plate. [3]

EasyOCR, PaddleOCR, and Tesseract represent the three main OCR engines that have excelled in accurately reading the characters on the license plate, while also balancing accuracy with language support and computational efficiency. [3] It is also important to note that the EasyOCR engine can skip the preprocessing step in reading the image, due to its strength in deep learning capabilities with reliable character recognition. EasyOCR performs this by using Character Region Awareness for Text Detection (CRAFT) which relies heavily on Convolutional Neural Networks (CNN's) followed by a Recurrent Neural Network (RNN) for extracting and recognizing the characters. [3] These findings were outlined in a 2024 preprint study by Tavares at the University of Goiás in Brazil, where they ultimately concluded that factors such as image distortion, low resolution, and variable lighting can impact which OCR engine is best suited for completing the important second step for any ALPR [3]

III. METHOD

A. Abbreviations and Acronyms

(ALPR) Automatic License Plate Recognition, (AMP) Automatic Mixed Precision, (CLAHE) Contrast Limited Adaptive Histogram Equalization, (CNN), Convolutional Neural Network, (COCO) Common Objects in Context, (CRAFT) Character Region Awareness for Text Detection, (CRNN) Convolutional Recurrent Neural Network, (CUDA) Compute Unified Device Architecture, (FP16) 16-bit Floating

Point, (GPU) Graphics Processing Unit, (IOU) Intersection Over Union, (mAP) mean Average Precision, (mAP50) mean Average Precision at 50% IOU threshold, (mAP50-95) mean Average Precision evaluated across IOU thresholds from 50% to 95%, (OCR) Optical Character Recognition, (regex) regular expression, (VRAM) Video Random Access Memory, (YAML) Yet Another Markup Language, (YOLO) You Only Look Once

B. The Dataset and Preprocessing

The dataset used for this study was sourced from Kaggle and consisted of 9,100 labeled vehicle images, split across three subsets: 7,052 training images, 2,048 validation images, and a reserved test partition. Before training began, the image files and their corresponding annotation labels, were converted into a YAML configuration file that the YOLO model accepted as its data definition. The YAML file was created with the help of a function which specified the path to the dataset root, with the relative paths to the training and validation image directories. It also used the number of object classes, corresponding to the license plate class, and the class name mapping. This configuration file served as the contract between the dataset and the YOLO training engine, ensuring consistent data loading across all training runs. Images were then scanned and cached prior to training, with five background images identified in the training set and three in the validation set. No corrupt images were detected in either partition.

To improve model efficiency, the Ultralytics training engine applied a suite of built-in data augmentation techniques during training, that included horizontal flip, mosaic composition, HSV color jitter, and random erasing. These augmentations exposed the model to a wider variety of plate appearances without requiring additional labeled data, simulating real-world variation in lighting, color, and partial occlusion which the research indicated could take place.

C. Model Architecture and Training Configuration

One of the python script's that was used for model training evaluated three YOLOv8 model variants: YOLOv8-Nano (YOLOv8n), YOLOv8-Small (YOLOv8s), and YOLOv8-Medium (YOLOv8m). Another script was developed to separate training configurations, which focused exclusively on YOLOv8n and was used to build the full ALPR pipeline, for batch inference and for real-world testing. After comparing the results of both approaches, the team decided to adopt the second script used for training and downstream pipeline as the primary system, as it produced stronger real-world performance and a more complete end-to-end implementation. It's worth reiterating, YOLO was selected as the detection architecture because license plate detection is an object localization problem. The system must identify not only that a plate is present, but also where it is located within the image. YOLO's single-stage detection approach processes the entire image in a single forward pass, returning bounding box coordinates and confidence scores simultaneously, which is well suited to the speed requirements of a real-time toll detection application.

The baseline training configuration used YOLOv8n initialized with pretrained COCO (Common Objects in Context) weights and fine-tuned on the license plate dataset.

Training was conducted for 30 epochs at an image resolution of 640 pixels and a batch size of 32 images. The 640-pixel image size is the standard resolution for YOLO-based detection tasks and balances spatial resolution with computational efficiency. The batch size of 32 was selected to fully utilize the available GPU (Graphics Processing Unit) memory on the NVIDIA GeForce RTX 3080 Ti that was used for most of the scripts.

A second training configuration extended training to 80 epochs to allow additional convergence time on the YOLOv8n model. Both runs used Automatic Mixed Precision (AMP) to leverage the GPU's Tensor Core architecture for accelerated FP16 (16-bit Floating Point) computation. The AdamW optimizer was selected automatically by the Ultralytics training engine, with a learning rate of 0.002 and a momentum of 0.9. Eight dataloader workers were used for parallel image loading, and a patience parameter was configured to allow early stopping if validation performance did not improve across consecutive epochs, reducing the risk of overfitting during longer training runs. All experiments were conducted on a workstation running Python 3.14 with PyTorch 2.12 and CUDA (Compute Unified Device Architecture) 12.6, accelerated by an NVIDIA GeForce RTX 3080 Ti GPU with 12GB of VRAM (Video Random Access Memory). The primary system applied a three-model comparison and used 5 initial epochs to confirm setup functionality, with image size set to 640 and batch size set to 8 to accommodate a wider range of hardware configurations. A confidence threshold of 0.25 was used for that evaluation, and a patience parameter was included for early stopping

D. OCR Pipeline

Following plate detection, the primary system passed each bounding box crop through a multi-stage OCR pipeline. Each detected plate region was extracted from the original image with a five-pixel padding border added to each edge to prevent edge characters from being excluded during the crop operation. Each crop was then upscaled by a factor of two using bicubic interpolation to improve character legibility for the OCR engine. To improve efficiency across varying image conditions, a three-pass preprocessing strategy was applied to each crop before OCR was performed. Pass 1 converted the crop to grayscale without further modification, preserving the raw luminance signal. Pass 2 applied CLAHE with a clip limit of 2.0 and an 8x8 tile grid to normalize contrast across regions of the plate, which is particularly effective for plates captured under uneven or low lighting conditions. Pass 3 applied an adaptive Gaussian threshold with a bitwise inversion step to produce a binarized image in which plate characters appear as light regions against a dark background, improving character segmentation for the OCR engine.

The result of Pass 3 on one of the team's test car plates is shown in Fig. 2, where the characters KPW 2002 are clearly isolated on a dark background following the adaptive threshold operation. EasyOCR was applied to each of the three preprocessed versions independently. An alphanumeric character allowlist restricting recognized output to the characters A through Z and digits 0 through 9 was enforced on all passes, eliminating symbol and punctuation detections. A regular expression (regex) heuristic targeting the standard New York

state license plate format, which consists of three uppercase letters followed by four digits and was applied to validate whether a recognizable plate pattern was present. A spatial centering gate filtered out any detected text whose horizontal center coordinate fell outside 90% of the plate crop's horizontal midpoint. The highest confidence valid match across all three preprocessing passes was selected as the final plate text output. The detection confidence threshold for all inference experiments was set to 0.40.

IV. RESULTS

A. YOLO Model Comparison

As part of the initial model evaluation, the first script trained and compared YOLOv8n, YOLOv8s, and YOLOv8m on the license plate dataset. The results of this comparison are shown in Figure 1, which presents precision, recall, mAP50, and mAP50-95 scores across all three models. Across all four metrics, YOLOv8s produced the strongest overall performance, achieving the highest mAP50 score of approximately 0.85 and a precision near 0.85. YOLOv8n followed closely with a mAP50 of approximately 0.82 and a precision near 0.83. YOLOv8m produced the lowest scores across all four metrics in this evaluation, which was an interesting result given that it is the largest of the three models tested. This outcome reinforces the point that a larger model is not always the best choice, and that speed, accuracy, and resource cost must all be weighed when selecting an architecture for real-time detection. The mAP50-95 scores for all three models fell in the range of 0.42 to 0.45. Based on this comparison, and on the stronger downstream pipeline performance observed in the alternate script, the team selected YOLOv8n as the model for the full ALPR system.

B. Detection Performance on the Training Run

The YOLOv8-Nano model trained by the primary system for 30 epochs demonstrated strong and consistent detection performance across the full training schedule. At epoch one, the model already produced competitive results with a precision of 0.742, a recall of 0.734, and an mAP50 of 0.696, reflecting the benefit of transfer learning from pretrained COCO weights. Performance improved steadily across subsequent epochs, with the most significant gains occurring in the first ten epochs as the classification loss decreased from 1.584 at epoch 1 to 0.566 at epoch 10. By epoch 30, the model reached a final validation precision of 0.984, recall of 0.944, mAP50 of 0.963, and mAP50-95 of 0.689. The full 30-epoch training run completed

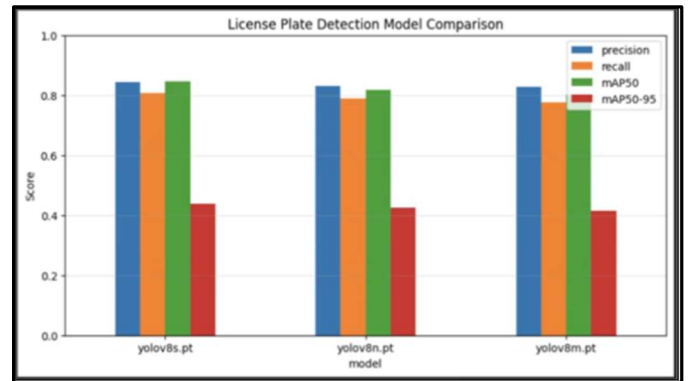


Fig. 1. License Plate Detection Model. (YOLO Comparison)

in approximately 16.5 minutes on the NVIDIA GeForce RTX 3080 Ti. Precision measures how often the model is correct when it predicts the presence of a license plate, and a high precision score indicates the model is producing few false detections. For a toll system application, high precision is especially important because the system should not mistake non-plate objects for chargeable vehicles. Recall measures how many real license plates the model successfully localizes, and a high recall score indicates the model is missing fewer plates in an image provided. The mAP50 (mean Average Precision at 50% Intersection Over Union) metric measures how well the predicted bounding box overlaps with the ground truth annotation at a 50% IOU threshold, and mAP50-95 extends this evaluation across overlap thresholds from 50% to 95%. The final mAP50-95 of 0.689 reflects that while the model locates plates reliably, there is room for improvement in bounding box tightness at stricter overlapping thresholds

C. Inference on Real-World Images

The trained YOLOv8-Nano model was evaluated against two real-world personal vehicle photographs, referred to as TylerCar and KimiCar, using a detection confidence threshold of 0.40. The model successfully detected one license plate per image in both cases, with an inference time of 21.4ms per image at a resolution of 640x480 pixels. This included 7.9ms for preprocessing and 5.8ms for postprocessing. (Fig. 2 below) These results confirm that the trained model generalizes to unseen vehicle images captured outside of the training distribution under real-world conditions, and that inference speed is well within the range required for a real-time toll or surveillance application.

D. OCR Performance and Observations

Unfortunately, when the full three-pass ALPR pipeline was applied to the two real-world test images, the OCR stage returned no valid pattern match for either image under the strict New York plate regex heuristic. This outcome indicates that while the YOLO detection stage performed correctly in localizing the plate regions in both images, the OCR stage did not produce a character string that conformed to the expected three-letter, four-digit New York plate format. Figure 3 illustrates a sample of OCR outputs produced during pipeline testing across multiple plate crops. Several crops returned no OCR text at all, particularly for low-contrast or dark images where character edges were not sufficiently resolved by any of the three preprocessing passes.



Fig. 2. YOLOv8-Nano Model Provided Image. (TylerCar Plate Detection)

TABLE I. YOLO COMPARISON

Model	YOLO Model Comparison Results			
	Precision	Recall	mAP50	mAP50-95
YOLOv8n	0.83	0.79	0.82	0.43
YOLOv8s	0.85	0.81	0.85	0.45
YOLOv8m	0.83	0.78	0.80	0.42

Other crops produced partial reads such as DRFA civil, Civil, and J0.64, reflecting cases where the OCR engine detected characters but produced partial text, mixed case output, or confused digits and letters. These examples illustrate the practical challenges of the OCR recognition stage, which Tavares (2024) noted is driven by variable lighting, low resolution, and perspective distortion in real-world ALPR deployments [3]. Figure 2 shows one case where the adaptive threshold preprocessing produced a clean, high-contrast binarized image of a New York plate reading KPW 2002, demonstrating that the preprocessing pipeline is capable of producing OCR-ready crops when image conditions are favorable. True OCR accuracy evaluation requires a dataset in which the ground truth plate text is labeled alongside the image. Because the test images were personal vehicle photographs without associated ground truth labels, a quantitative OCR accuracy metric could not be computed. Future iterations of this system could expand the regex heuristic to support plate formats from multiple states or countries, incorporate PaddleOCR or Tesseract as comparative OCR engines, and extend the pipeline to video input for frame-by-frame plate tracking.

V. CONCLUSION

This study demonstrated that a two-stage ALPR pipeline combining YOLOv8-Nano detection with EasyOCR character recognition is capable of achieving strong license plate detection performance on both a structured validation dataset and unseen real-world vehicle images. The YOLOv8-Nano model achieved a mAP50 of 0.963 and a precision of 0.984 after 30 epochs of training, completing in under 17 minutes on consumer GPU hardware. The three-model comparison showed that model size alone does not determine detection accuracy, as YOLOv8s outperformed YOLOv8m in this evaluation. The OCR stage highlighted the practical difficulty of character recognition under real-world conditions, where lighting, resolution, and plate format variation reduce the effectiveness of pattern-based heuristics. Future work should include formal OCR accuracy evaluation using ground truth labeled plates, video pipeline testing to simulate toll booth conditions, expansion of the regex heuristic to support multiple state plate formats, and comparison of EasyOCR against PaddleOCR and Tesseract as alternative recognition engines.



Fig. 3. OCR Performance (OCR Results)

ACKNOWLEDGEMENT

We would like to thank Syracuse University and Professor Syed for their guidance in completing this project.

REFERENCES

- [1] H. Moussaoui, N. El Akkad, M. Benslimane, W. El-Shafai, A. Baihan, C. Hewage, and R. S. Rathore, "Enhancing automated vehicle identification by integrating YOLO v8 and OCR techniques for high-precision license plate detection and recognition," *Scientific Reports*, vol. 14, 2024. [Online]. Available: <https://www.nature.com/articles/s41598-024-65272-1>
- [2] T. A. Shyaa and A. A. Hashim, "Superior use of YOLOv8 to enhance car license plates detection speed and accuracy," *Revue d'Intelligence Artificielle*, vol. 38, no. 1, pp. 139-145, Feb. 2024. [Online]. Available: <https://doi.org/10.18280/ria.380114>
- [3] R. A. Tavares, "Comparison of image preprocessing techniques for vehicle license plate recognition using OCR: Performance and accuracy evaluation," *Universidade Federal de Goias, Goiania, Brazil*, arXiv preprint arXiv:2410.13622, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.13622>
- [4] A. Shamim, "License Plate Recognition Dataset," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/adilshamim8/license-plate-recognition>